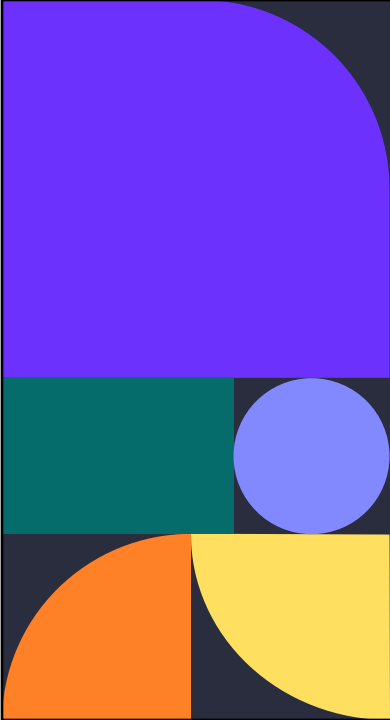


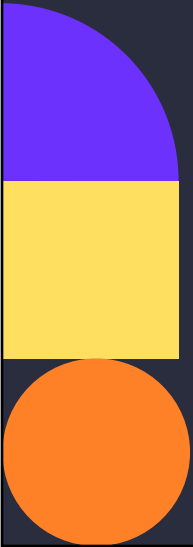


Individual Project Instructions and Examples

Merve Tekgürler

06/19/2025

1



Contents

- 01. **Assignment Description**
- 02. **Finding Code/Projects**
- 03. **Example**

2

Individual Project

The purpose of the individual project is to build necessary analytical skills to study the code and digital approaches behind various projects. It consists of two parts:

1. Short Paper
2. Jupyter Notebook Presentation

Motivations

To build a strong foundation in digital humanities, it is essential to become familiar with how digital humanists use Python in their research and teaching. This involves studying and analyzing code written by other researchers, as well as reproducing their workflows to understand both the technical and interpretive dimensions of their work. In doing so, one should also critically evaluate the digital humanities projects under study—examining their data sources, methodological approaches, design choices, and coding decisions—to assess their scholarly rigor, transparency, and impact.

Short Paper:

Due June 30, 2025

You will choose a topic, either a methodology/approach like topic modeling or a task, like named entity recognition and identify **three papers** on your chosen topic. You will briefly summarize their methods, why they chose these methods, and study their code and data. You will critically evaluate the papers to understand where the weaknesses and strengths of these papers are. Your paper should be 2 pages long.



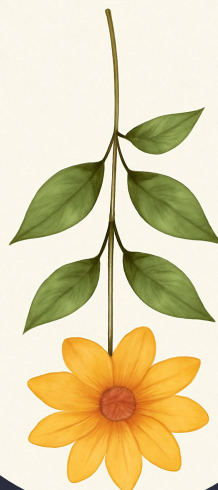
5

Code Presentation

Due June 30, 2025

You will replicate the code in one of the papers you evaluated and walk us through that code. In order to do that, you need to create a **Jupyter Notebook** and get the research code running in our dighum101 environment. You will submit the Jupyter Notebook on bCourses and present it in class during Week 6. Your presentations should be 5-6 mins long.

You have the option of screen recording your presentation and sharing it with me ahead of time, which might be useful especially for those who have internet access issues. If this is the case, please let me know ahead of time. You are still required to be present in class.



6

Reproducing code deepens understanding, sharpens technical skills, and reveals research methodologies.

7

Where and how to find code and projects?

8

Resources



General

<https://aclanthology.org/>
<https://www.nature.com/palco>
mms/
<https://paperswithcode.com/>
<https://zenodo.org/>
<https://scholar.google.com/>
<https://www.semanticscholar.org/>

Digital Humanities

<https://www.digitalhumanities.org/dha/>
<https://eupublishing.com/loi/ijhac>
<https://openhumanitiesdata.me/tajnl.com/>
<https://academic.oup.com/ds>
<https://culturalanalytics.org/>
<https://journalofdigitalhistory.org/>



Example

Short Paper:

This is a mini *literature review*

General problem/task definition: What are these papers trying to solve, and why?

Concise summaries of the articles: Do not simply copy the article text in full. We can read them ourselves. Put in your own words the major contributions of each article.

Compare and contrast: Point out the similarities and differences of the papers. Do they agree with each other? Are results seemingly in conflict? If the papers address different subtasks, how are they related? (If they are not related, then you may have made poor choices for a lit review...)

Future work: Make several suggestions for how the work can be extended. Are there open questions to answer? How do the papers relate to your final project idea?

References section: The entries should appear alphabetically and give at least full author name(s), year of publication, title, and outlet if applicable (e.g., journal name or proceedings name). References are not a part of the page count.



Review Article

Defeng Li*, Kan Wu and Victoria L.C. Lei

Applying Topic Modeling to Literary Analysis: A Review

<https://doi.org/10.1515/dsll-2024-0010>

Received August 18, 2024; accepted November 6, 2024; published online November 26, 2024

Abstract: This article examines the integration of topic modeling within literary studies, highlighting its potential to transform conventional literary analysis through computational techniques. It reviews the theoretical underpinnings underlying topic modeling, including prominent algorithms like Latent Dirichlet Allocation, Non-negative Matrix Factorization, and Neural Topic Models, and discuss their utility in dissecting large textual corpora to uncover latent thematic and stylistic patterns. The article subsequently addresses the specific methodological steps for effective implementation, spanning text preprocessing model tuning and result interpretation. We further illustrate the diverse applications of topic modeling in literary studies through thematic analysis, comparative studies, and the extraction of cultural and historical insights. Challenges such as model accuracy, technical limitations, and ethical considerations are critically assessed. The review concludes by envisioning prospective future directions that foresee enhanced integration of topic modeling into literary criticism, facilitated by technological and interdisciplinary advancements.

Keywords: topic modeling; literary studies; computational literary analysis; latent dirichlet allocation; textual data analysis

<https://doi.org/10.1515/dsll-2024-0010>

Code Reproduction:

This is both a Jupyter Notebook submission and a presentation

Your goal here is not to do something new but to reproduce a piece of code that is interesting for you. You will learn a lot about how this code works and how you can adapt it to your work.



13

arXiv > cs > arXiv:2203.05794

Search...
Help | Ad

Computer Science > Computation and Language

[Submitted on 11 Mar 2022]

BERTopic: Neural topic modeling with a class-based TF-IDF procedure

Maarten Grootendorst

Topic models can be useful tools to discover latent topics in collections of documents. Recent studies have shown the feasibility of approach topic modeling as a clustering task. We present BERTopic, a topic model that extends this process by extracting coherent topic representation through the development of a class-based variation of TF-IDF. More specifically, BERTopic generates document embedding with pre-trained transformer-based language models, clusters these embeddings, and finally, generates topic representations with the class-based TF-IDF procedure. BERTopic generates coherent topics and remains competitive across a variety of benchmarks involving classical models and those that follow the more recent clustering approach of topic modeling.

Comments: BERTopic has a python implementation, see this [https URL](https://github.com/MaartenGrootendorst/BERTopic)

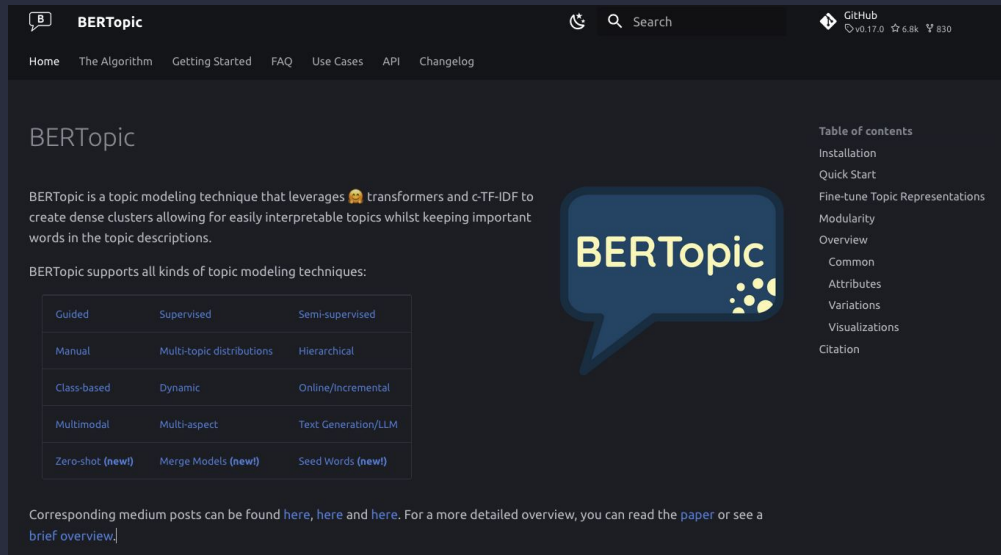
Subjects: **Computation and Language (cs.CL)**

Cite as: arXiv:2203.05794 [cs.CL]
(or arXiv:2203.05794v1 [cs.CL] for this version)
<https://doi.org/10.48550/arXiv.2203.05794>

Submission history

From: Maarten Grootendorst [[view email](#)]
[v1] Fri, 11 Mar 2022 08:35:15 UTC (384 KB)

<https://arxiv.org/abs/2203.05794>

The image shows the GitHub repository page for BERTopic. The header includes the repository name 'BERTopic', a search bar, and GitHub statistics (0.17.0, 6.8k stars, 830 forks). The navigation menu lists 'Home', 'The Algorithm', 'Getting Started', 'FAQ', 'Use Cases', 'API', and 'Changelog'. The main content area features the title 'BERTopic', a description of the technique, and a table of supported modeling techniques. A 'Table of contents' sidebar is on the right.

BERTopic

BERTopic is a topic modeling technique that leverages 🧠 transformers and c-TF-IDF to create dense clusters allowing for easily interpretable topics whilst keeping important words in the topic descriptions.

BERTopic supports all kinds of topic modeling techniques:

Guided	Supervised	Semi-supervised
Manual	Multi-topic distributions	Hierarchical
Class-based	Dynamic	Online/Incremental
Multimodal	Multi-aspect	Text Generation/LLM
Zero-shot (new!)	Merge Models (new!)	Seed Words (new!)

Corresponding medium posts can be found [here](#), [here](#) and [here](#). For a more detailed overview, you can read the [paper](#) or see a [brief overview](#).

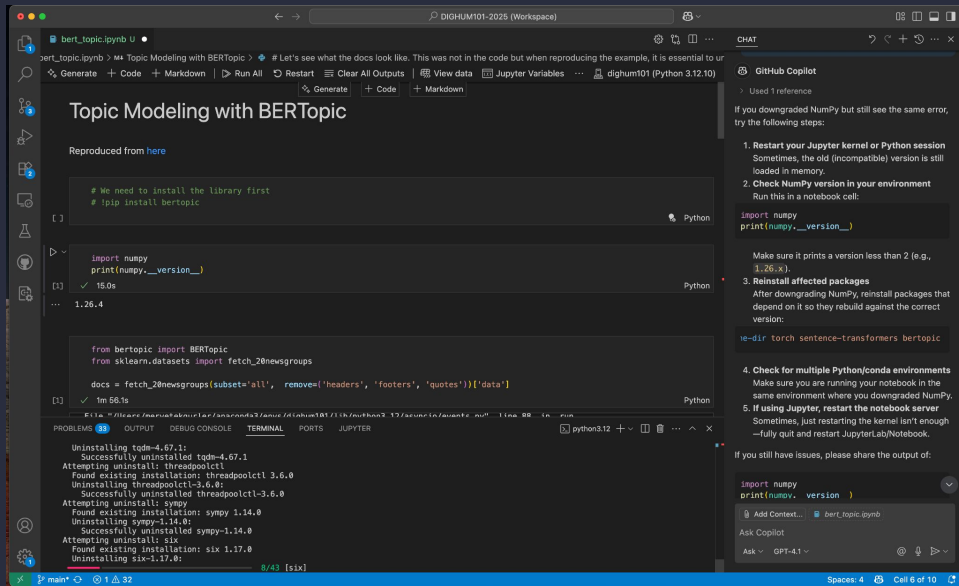
Table of contents

- Installation
- Quick Start
- Fine-tune Topic Representations
- Modularity
- Overview
 - Common
 - Attributes
 - Variations
 - Visualizations
- Citation

<https://maartengr.github.io/BERTopic/index.html>

Next Steps

1. Open your VS Code and create a new Jupyter Notebook
2. Start installing necessary libraries and dependencies
3. Bring in any datasets that you need to run the code
4. Copy paste the code from the original source
5. Run it cell-by-cell, where necessary add explanations and even more code of your own. You want to see what is in the dataset, even if the author of the code does not have a line that explicitly shows it.



GitHub Copilot can help troubleshoot

Any questions?

Let's talk more on Monday during my office hours.